
Lifeline: Offline Peer-to-Peer Emergency SOS

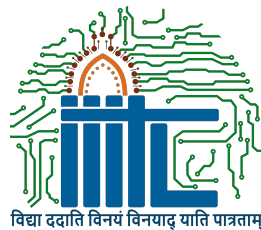
*A project report submitted in partial fulfillment of the requirements for the
award of the degree of*

B.Tech. in Computer Science

by

Prateek Mishra (LCB2023031)
Prashik Hemraj Humane (LCB2023039)
Mayank (LIT2023008)
Arun Kumar Swami (LCB2023049)
Eshant Gupta (LCB2023016)

under the guidance of
Sushil Kumar Tiwari



Indian Institute of Information Technology, Lucknow
June 2026

© Indian Institute of Information Technology, Lucknow 2026.

Declaration of Authorship

I/we, **Prateek Mishra, Prashik Hemraj Humane, Mayank, Arun Kumar Swami, and Eshant Gupta**, declare that the work presented in “**Lifeline: Offline Peer-to-Peer Emergency SOS**” is my/our own. I/we confirm that:

- This work was completed entirely while in candidature for B.Tech. degree at Indian Institute of Information Technology, Lucknow.
- Where I/we have consulted the published work of others, it is always cited.
- Wherever I/we have cited the work of others, the source is always indicated. Except for the aforementioned quotations, this work is solely my/our work.
- I have acknowledged all major sources of information.

Signed:

<i>P. Mishra</i>	<i>P. Humane</i>	<i>Mayank</i>	<i>A. Swami</i>	<i>Eshant G.</i>
(Prateek)	(Prashik)	(Mayank)	(Arun)	(Eshant)

Date: **4 June 2026**

CERTIFICATE

This is to certify that the work entitled “**LIFELINE: OFFLINE PEER-TO-PEER EMERGENCY SOS**” submitted by **Prateek Mishra, Prashik Hemraj Humane, Mayank, Arun Kumar Swami, and Eshant Gupta** who got their names registered on **Jul 2021** for the award of B.Tech. degree at Indian Institute of Information Technology, Lucknow is absolutely based upon their own work under the supervision of **Sushil Kumar Tiwari**, Department of Computer Science, Indian Institute of Information Technology, Lucknow - 226 002, U.P., India and that neither this work nor any part of it has been submitted for any degree/diploma or any other academic award anywhere before.

Sushil Kumar Tiwari
(Supervisor)

Department of Computer Science
Indian Institute of Information Technology, Lucknow
Pin - 226002, INDIA

Acknowledgements

We would like to express our deepest gratitude and sincere thanks to our supervisor, **Sushil Kumar Tiwari**, for his invaluable guidance, constant support, and encouragement throughout the course of this project. His expertise and constructive feedback were instrumental in shaping the "Life-line" application and helping us overcome various technical challenges.

We also extend our heartfelt thanks to the Department of Computer Science at the Indian Institute of Information Technology, Lucknow, for providing the necessary resources, infrastructure, and an environment conducive to learning and research.

Finally, we are immensely grateful to our families and friends for their unwavering support, motivation, and patience during our B.Tech journey.

Lucknow
June 2026

Prateek Mishra
Prashik Hemraj Humane
Mayank
Arun Kumar Swami
Eshant Gupta

ABSTRACT

During severe natural disasters such as earthquakes, floods, or cyclones, traditional communication infrastructure like cellular towers and internet service providers frequently fail, leaving affected individuals completely isolated. In such critical scenarios, the inability to broadcast a distress signal can result in a delayed rescue response and loss of life.

This project introduces **Lifeline**, a highly resilient Android application designed to facilitate offline peer-to-peer emergency communication. Lifeline enables users to broadcast distress signals (SOS) completely independent of a SIM card or internet connection. By utilizing the Google Nearby Connections API, the application forms a decentralized mesh network across smartphones using Bluetooth Low Energy and WiFi Direct. As devices come into proximity, SOS messages are silently relayed from one device to another, continuously expanding the rescue radius.

The application further incorporates offline map capabilities using 'osmdroid', allowing rescuers to pinpoint distress signals geographically without requiring an active data connection. Finally, if any node within the mesh network eventually acquires internet access, Lifeline automatically syncs the collected SOS signals to a centralized Firebase Firestore database, enabling remote authorities to assess the disaster impact in real-time. This project successfully demonstrates a functional proof-of-concept for a reliable, offline-first emergency communication system.

Contents

1	Introduction	1
1.1	Overview	1
1.2	Problem Statement	1
1.3	Motivation	2
1.4	Objective	2
1.5	Scope of the Project	2
1.6	Report Organization	3
2	Literature Review and Background	4
2.1	Traditional Emergency SOS Systems	4
2.2	Comparison: Traditional Apps vs Lifeline	4
2.3	The Concept of Mesh Networking	4
2.4	Google Nearby Connections API	5
2.4.1	How It Works	5
3	System Design and Methodology	7
3.1	System Architecture	7
3.1.1	The MVVM Components	7
3.2	Core Technologies Stack	8
3.3	Application Data Flow	8
4	Implementation and Core Features	10
4.1	Offline SOS Broadcasting	10
4.2	Audio SOS Messages	10
4.3	Offline Rescue Maps	11
4.4	Use Case Scenario: The Earthquake Protocol	11
4.5	Background Cloud Synchronization	12
5	Conclusion and Future Scope	13
5.1	Conclusion	13
5.2	Impact and Applicability	13

5.3	Future Scope	14
-----	------------------------	----

Chapter 1

Introduction

1.1 Overview

Natural disasters such as earthquakes, tsunamis, and severe storms often result in the immediate and catastrophic collapse of local communication infrastructure. During these events, cellular towers lose power, fiber optic cables are severed, and internet service providers experience prolonged downtimes due to physical damage or sudden traffic congestion. In these critical hours, affected individuals are left completely isolated, unable to call for help, notify their families of their safety, or coordinate with search and rescue teams. The "Lifeline" project aims to address this critical communication gap by providing a reliable, offline-first platform that operates entirely independently of traditional network infrastructures.

1.2 Problem Statement

During and immediately following a disaster, the inability to broadcast a distress signal significantly hinders search and rescue operations. Traditional SOS applications and built-in mobile features rely exclusively on SMS or active mobile data connections to function. When the cellular grid fails, these applications are rendered completely useless. Victims find themselves trapped under debris or isolated by floodwaters without any means of signaling their precise location or status to the outside world, drastically reducing their chances of a timely rescue.

1.3 Motivation

The primary motivation behind this project stems from analyzing the communication failures during recent global natural disasters. Time is the most critical factor in rescue operations; the "golden hours" immediately following an incident dictate survival rates. We realized that while modern smartphones are incredibly powerful devices equipped with diverse radio technologies (Bluetooth, WiFi), they become practically useless "bricks" when disconnected from the internet. Lifeline was conceived to unlock the latent potential of these devices by allowing them to communicate directly with one another.

1.4 Objective

The primary objective of this project is to design, develop, and test an Android application capable of broadcasting and receiving emergency SOS signals without requiring a SIM card, cellular network, or active internet connection.

To achieve this, the application constructs a decentralized peer-to-peer (P2P) mesh network using the physical Bluetooth and WiFi Direct radios present in modern smartphones. This allows the application to relay distress signals invisibly across a chain of nearby devices until the signal reaches a rescuer or a node with an active internet connection.

1.5 Scope of the Project

The project encompasses the following core areas and deliverables:

- **Offline Communication Protocol:** Implementation of a decentralized, multi-hop communication system using the Google Nearby Connections API.
- **SOS Data Broadcasting:** Development of an offline SOS broadcast system supporting the transmission of exact GPS coordinates, user safety status (SAFE/TRAPPED), and custom voice data.
- **Geographic Visualization:** Integration of an offline map viewer powered by 'osmdroid' to visually pinpoint distress signals on a map without requiring internet access.

- **Cloud Integration:** Design of a background synchronization mechanism (using Android WorkManager) to upload collected SOS data to a cloud database (Firebase Firestore) whenever any node in the mesh network regains internet access, ensuring remote authorities can access the data.

1.6 Report Organization

The remainder of this report is organized as follows: Chapter 2 discusses the literature review and the background of mesh networking. Chapter 3 outlines the system design, architecture, and core technologies utilized. Chapter 4 provides a detailed explanation of the implementation and core features of the application. Finally, Chapter 5 concludes the report and discusses the future scope of the project.

Chapter 2

Literature Review and Background

2.1 Traditional Emergency SOS Systems

Currently available emergency applications heavily depend on cellular services. When a user triggers an SOS, the system typically sends an SMS containing their GPS coordinates to pre-selected contacts, or it initiates a VoIP call to emergency services over mobile data. While effective in normal conditions, these systems exhibit a single point of failure: the cellular network. During large-scale disasters, cellular towers are often destroyed, stripped of power, or simply overwhelmed by a sudden surge in traffic as thousands of people attempt to call their loved ones simultaneously. This renders traditional SOS applications entirely ineffective just when they are needed most.

2.2 Comparison: Traditional Apps vs Lifeline

To clearly illustrate the paradigm shift, Table 2.1 outlines the core differences between traditional emergency applications and the Lifeline system.

2.3 The Concept of Mesh Networking

A mesh network is a local network topology in which the infrastructure nodes (in this case, smartphones) connect directly, dynamically, and non-hierarchically to as many other nodes as possible. Instead of all devices

Feature	Traditional SOS Apps	Lifeline App
Network Required	Cellular Network, Internet	None (Offline P2P)
Hardware Reliance	Telecom Towers	Standard Smartphone Radios
Disaster Resilience	Low (Fails on grid collapse)	High (Independent of grid)
Range Limit	Global (if connected)	Local Mesh (Expandable via hops)
Voice Support	High-bandwidth VoIP	Low-bandwidth offline audio bytes

Table 2.1: Comparison of Emergency Communication Systems

talking to a central cellular tower, they cooperate with one another to efficiently route data from and to clients.

This decentralized structure is the backbone of Lifeline. It allows the network to remain operational even when an individual node or connection fails. Lifeline adopts this concept by turning everyday smartphones into individual relay nodes that can securely pass distress signals along a chain of devices until the signal physically moves out of the disaster zone and reaches help.

2.4 Google Nearby Connections API

To facilitate the creation of this offline mesh network without requiring users to manually configure their Bluetooth settings, the Lifeline application utilizes the **Google Nearby Connections API**.

2.4.1 How It Works

This API allows applications to discover, connect to, and exchange data with nearby devices in real-time. Under the hood, it abstracts the immense complexity of raw radio technologies and intelligently switches between:

- **Bluetooth Low Energy (BLE):** Used for highly energy-efficient, continuous background discovery of nearby devices.
- **Classic Bluetooth:** Used for establishing initial connections and sending small payloads (like text SOS messages and GPS coordinates).

- **WiFi Direct:** Automatically upgraded to when a high-bandwidth connection is required (such as when transmitting a recorded voice note).

Lifeline uses the "Cluster" strategy of this API. The Cluster strategy is specifically designed to create small, dynamic, and decentralized networks of peer devices, making it the perfect choice for ad-hoc disaster communication.

Chapter 3

System Design and Methodology

3.1 System Architecture

The Lifeline application follows the robust Model-View-ViewModel (MVVM) architectural pattern recommended by Google for modern Android development. This architecture ensures a strict separation of concerns, making the application stable, scalable, and easy to maintain by a team of developers.

3.1.1 The MVVM Components

- **Model (Data Layer):** This layer is completely isolated from the UI. It contains the data structures (e.g., 'MessageEntity' which defines the schema for an SOS message) and the Room Database repositories. The repositories act as the single source of truth, handling all local storage operations and deciding when to fetch or sync data.
- **ViewModel:** The ViewModel acts as the crucial bridge between the Model and the View. It manages the UI-related data in a lifecycle-conscious way (meaning data isn't lost during screen rotations). It also contains the business logic to interact with the Nearby Connections API, initiating discovery and managing active peer connections.
- **View (UI Layer):** The UI layer consists of the XML layouts and Kotlin Activities/Fragments. It strictly observes the ViewModel for state changes and updates the screen accordingly (e.g., updating the map when a new SOS coordinate is received).

3.2 Core Technologies Stack

To fulfill the complex offline and synchronization requirements of the project, we carefully selected a modern technology stack:

- **Kotlin:** The primary, statically-typed programming language used for Android development, offering null-safety and coroutines for asynchronous programming.
- **Google Nearby Connections API:** The engine driving our offline mesh network, handling device discovery, connection handshakes, and payload transfers over Bluetooth and WiFi.
- **Room Database (SQLite):** A powerful local persistence library that safely stores incoming and outgoing SOS messages directly on the device's internal storage, even when the app is closed.
- **osmdroid:** A highly customizable, open-source map viewer. Unlike Google Maps, osmdroid explicitly supports offline tile caching, allowing rescuers to view detailed geographic data completely without an internet connection.
- **Firebase Firestore:** A NoSQL cloud database used to synchronize the locally collected SOS messages. It acts as the centralized command center for remote authorities.
- **Android WorkManager:** A reliable background task scheduler used to quietly sync data to Firebase whenever the device connects to the internet, without draining the battery.

3.3 Application Data Flow

The logic of the application is designed to be fully asynchronous and non-blocking. The data flow proceeds as follows:

1. **Initiation:** A user initiates an SOS broadcast by pressing the emergency button on the dashboard.
2. **Data Capture:** The application queries the device's GPS hardware to capture the exact latitude and longitude. The user can also append a text status or a recorded voice note.
3. **Local Storage:** The composed message is immediately saved to the local Room database to ensure it isn't lost if the app crashes.

4. **Broadcasting:** The Nearby Connections API packages this data into a secure byte payload and broadcasts it to all connected peers in the local mesh cluster.
5. **Re-broadcasting (The Mesh Effect):** When receiving devices acquire the payload, they store it locally and instantly re-broadcast it to **their** connected peers, expanding the physical range of the SOS signal exponentially.
6. **Cloud Sync:** Finally, if any device containing these stored SOS messages connects to a cellular tower or WiFi with internet access, the WorkManager task activates and uploads the entire chain of data to Firebase Firestore for remote access by emergency services.

Chapter 4

Implementation and Core Features

4.1 Offline SOS Broadcasting

The core function of Lifeline is to empower users to broadcast a distress signal effortlessly, even under extreme duress. The application features a prominent, highly visible SOS button on the main dashboard.

Upon activation, the application queries the device's native 'Location-Manager' and 'FusedLocationProviderClient'. It aggressively attempts to retrieve the most accurate GPS coordinates available. Once locked, these coordinates—along with the user's declared status ("SAFE" or "TRAPPED"), their emergency contact information, and an optional custom text message—are serialized into a lightweight byte array payload and continuously broadcasted to the local environment.

4.2 Audio SOS Messages

During building collapses or severe injuries, a user may be physically incapable of typing a detailed message. To provide crucial context to rescuers, the application allows users to record brief audio messages using the device's microphone.

Because audio files are significantly larger than text coordinates, they cannot be efficiently sent over Bluetooth Low Energy. Therefore, Lifeline leverages the Nearby Connections API to automatically negotiate a high-bandwidth WiFi Direct channel between peers. Once established, the audio bytes are transmitted at high speeds. Upon receiving the audio bytes,

nearby devices can instantly play back the voice message, allowing rescuers to hear the victim's exact situation.

4.3 Offline Rescue Maps

A distress signal is useless if rescuers cannot navigate to the location. Since Google Maps and other commercial mapping services require an active internet connection to load tiles, they fail during disasters.

Lifeline incorporates a robust offline map feature powered by the 'osmdroid' library.

- **Pre-Caching:** Before venturing into disaster-prone areas, users or rescue workers can cache the map tiles of a specific region into their device's local storage.
- **Live Plotting:** During an emergency, when distress signals (containing latitude and longitude) are received via the mesh network, the application plots these signals as distinct red markers directly on the cached offline map.

This visual representation allows peers and rescuers to identify the exact geographic location of trapped individuals and navigate towards them without relying on a cellular data connection.

4.4 Use Case Scenario: The Earthquake Protocol

To understand the practical application of Lifeline, consider the following scenario:

1. A massive earthquake strikes a city, instantly bringing down all cellular towers.
2. Victim A is trapped in a basement. They open Lifeline and press the SOS button.
3. The app fetches their GPS location and begins broadcasting via Bluetooth.
4. Person B, walking on the street 50 meters away, has Lifeline installed. Their phone silently picks up Victim A's signal, stores it, and begins re-broadcasting it.

5. Rescue Worker C, 150 meters away, picks up the signal from Person B. The offline map on Worker C's phone plots a red marker showing exactly where Victim A is buried.

4.5 Background Cloud Synchronization

While the primary focus is offline, localized communication, Lifeline also bridges the gap to the outside world. It features a 'SyncWorker' built on top of Android's 'WorkManager'.

This background service periodically pings for an active internet connection. If a connection is detected (e.g., a rescue worker leaves the disaster zone and enters a functioning city), the service automatically uploads all locally stored SOS messages from the mesh network to a central Firebase Firestore database. This ensures that the entire chain of distress signals is backed up and made available in real-time to remote government authorities and disaster management teams across the globe.

Chapter 5

Conclusion and Future Scope

5.1 Conclusion

The "Lifeline" project successfully demonstrates the immense feasibility and critical importance of decentralized, offline-first communication systems during severe emergencies. By ingeniously leveraging the standard radios (Bluetooth Low Energy and WiFi Direct) already present in billions of modern smartphones, the application circumvents the traditional telecommunications grid.

Through the Google Nearby Connections API, Lifeline creates a robust, self-healing peer-to-peer mesh network capable of routing SOS distress signals across vast distances without any centralized cellular infrastructure. The integration of offline mapping using 'osmdroid' provides immediate, actionable geographic data to local peers and rescuers. Furthermore, the seamless background synchronization mechanism with Firebase Firestore ensures that the localized mesh data is ultimately delivered to global disaster management platforms once internet connectivity is restored at the edge of the disaster zone.

Overall, Lifeline serves as a highly functional, scalable, and life-saving proof-of-concept for modern disaster resilience and search-and-rescue operations.

5.2 Impact and Applicability

The potential impact of deploying such a system is profound. In scenarios like the 2023 Turkey-Syria earthquake or massive floods where victims were trapped for days without network access, a system like Lifeline could have passively aggregated the locations of thousands of victims, provid-

ing search and rescue teams (NDRF, FEMA) with a digital "heat map" of where to focus their excavation efforts, saving countless lives and optimizing resource allocation.

5.3 Future Scope

While the current Android implementation successfully covers the fundamental requirements of an offline SOS system, several technical enhancements can be pursued in the future to solidify its effectiveness:

- **Cross-Platform Integration (iOS):** Expanding the mesh network to include iOS devices using Apple's 'MultipeerConnectivity' framework. Bridging the Android and iOS networks would double the potential node density in any given disaster scenario.
- **Hardware Radio Expansion (LoRa):** Integrating the application with external LoRa (Long Range) radio modules via a standard Bluetooth connection. While WiFi Direct caps out at a few hundred meters, LoRa modules can transmit small SOS packets over several kilometers, dramatically increasing the mesh network's coverage area.
- **Smart Battery Optimization:** Implementing adaptive broadcast intervals (e.g., broadcasting less frequently if the battery drops below 15%) to conserve the trapped user's device battery during prolonged entrapments that stretch into multiple days.